



Asymptotic Notations

SM Ariful Hoque
Lecturer
Jashore University of Science and Technology



What is it?

In computer science, **asymptotic notations** provide a way to describe how the running time or space requirements of an algorithm scale with the input size.

Help us understand the "**big picture**" of an algorithm's efficiency, particularly as the size of the input grows very large.



Why Use Asymptotic Notations?

Asymptotic notations allow us to:

1. Abstract Away Constants and Lower Order Terms
2. Compare Algorithm Performance



Big O Notation (O)

Big O notation provides an upper bound on the growth rate of an algorithm's running time or space requirements. It tells us the worst-case scenario for how an algorithm's performance scales with the input size.



Big O Notation (O)

Definition: If a function $f(n)$ is $O(g(n))$, there exists a constant c and a threshold n_0 such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$

Intuition: Think of Big O as the ceiling function; it defines the maximum "speed" the function $f(n)$ can grow.



Big Omega Notation (Ω)

Big Omega notation provides a **lower bound** on the growth rate of an algorithm. It tells us the **best-case scenario** for how an algorithm performs as the input size increases.

- **Definition:** If a function $f(n)$ is $\Omega(g(n))$, there exists a constant c and threshold n_0 such that for all $n \geq n_0$, $f(n) \geq c \cdot g(n)$
- **Intuition:** Think of Big Omega as the floor function, giving us the best possible growth rate of an algorithm.



Theta Notation (Θ)

Theta notation provides a tight bound on the growth rate of an algorithm. It tells us that an algorithm's growth rate is sandwiched between an upper and lower bound, both of which are asymptotically the same.



Theta Notation (Θ)

What should be the definition of theta??



Little o Notation (o)

Little o notation describes an **upper bound that is not asymptotically tight**. In other words, it's like Big O but indicates that $f(n)$ grows strictly slower than $g(n)$.

- **Definition:** If $f(n) = o(g(n))$, for every constant $c > 0$ there exists an n_0 such that $f(n) < c \cdot g(n)$ for all $n > n_0$.
- **Intuition:** This notation shows that $f(n)$ grows slower than $g(n)$ but without reaching it asymptotically.



Little Omega Notation (ω)

Little Omega notation describes a **lower bound that is not asymptotically tight**. This means $f(n)$ grows strictly faster than $g(n)$ as n increases.

- **Definition:** If $f(n) = \omega(g(n))$, for every constant $c > 0$, there exists an n_0 such that $f(n) > c \cdot g(n)$ for all $n > n_0$
- **Intuition:** Little Omega shows that $f(n)$ grows faster than $g(n)$ but without reaching the same asymptotic growth.